

Aula 07 - Atividade

SIMULADOR DE CONDICIONAMENTO ELÉTRICO E NÍVEIS LÓGICOS

Ambiente: Compilador C Online (onlinegdb.com) — Linguagem C

Objetivo: Simular o processamento de sinais analógicos de sensores (Aula 07) e a lógica de decisão de um microcontrolador (Aula 06), sem necessidade de placa física.

1. Parâmetro Individual

Para validar sua entrega, utilize o **último dígito da sua matrícula (D)**. (Se o último dígito for 0, adote $D = 5$).

- **Corrente Nominal do LED (I_{LED}):** $I_{LED} = 10 + D$ (em mA)
(Exemplo: se a matrícula termina em 4, $I_{LED} = 14mA = 0,014A$)
- **Tensão da Fonte (V_{CC}):** 5,0V
- **Tensão do LED (V_L):** 2,0V

2. Guia Didático da Atividade: Do Esqueleto ao Resultado Final

A Situação Atual do Código (O que acontece se você rodar agora?) O arquivo `aula07code.c` foi entregue como um esqueleto estruturado de firmware em Linguagem C. Se você colar o código no OnlineGDB e clicar em **Run** imediatamente, o programa **não vai compilar** e emitirá mensagens de erro:

- **error: 'rLED' undeclared:** o comando `printf` tenta exibir o resistor calculado, mas a variável `rLED` ainda não existe no código.
- **error: 'valorADC' undeclared:** o laço de repetição tenta imprimir o valor digitalizado do sensor, mas a variável `valorADC` ainda não foi criada.

Toda a infraestrutura do sistema já está pronta (as diretivas `#define`, o menu `enum`, o vetor com as 4 tensões de teste e a máquina de decisão `switch...case`). O programa está apenas esperando que você crie as variáveis e escreva as fórmulas matemáticas que faltam.

A Sua Tarefa (Quais alterações você deve fazer no código?) Você deve localizar os três comentários marcados com `// TODO` e realizar exatamente as seguintes modificações:

- **1. Inserir seu Dígito de Matrícula:**
 - Altere `int ultimoDigito = 0;` substituindo o zero pelo **último número da sua matrícula**.
 - (Exemplo: se a matrícula termina em 7, escreva `int ultimoDigito = 7;`. Se a sua matrícula terminar em 0, o comando `if` logo abaixo já ajustará automaticamente para 5).
- **2. Implementar a Lei de Ohm:**
 - Logo abaixo do comentário `// TODO`, declare a variável `float rLED` e aplique a fórmula da Lei de Ohm para descobrir a resistência necessária para o LED:

C

```
float rLED = (TENSAO_FONTE - TENSAO_LED) / correnteLED_A;
```

- **3. Implementar a Conversão A/D de 10 bits (Linha 44):**

- Dentro do laço **for**, logo abaixo do comentário **// TODO**, declare a variável **uint16_t valorADC** e converta a tensão analógica **vin** para a escala digital de 0 a 1023 LSB do microcontrolador:

C

```
uint16_t valorADC = (uint16_t)round((vin / TENSAO_FONTE) * 1023.0f);
```

O Que Deve Aparecer no Resultado Final (Console de Execução)

Após fazer as 3 alterações e clicar em **Run**, o código compilará com sucesso e exibirá no terminal preto do OnlineGDB duas etapas automáticas:

1. **Cabeçalho de Inicialização:** Apresenta a identificação da sua matrícula e o resistor limitador calculado sob medida para a sua corrente nominal.
2. **Processamento das 4 Amostras de Tensão:** O laço testa 4 cenários elétricos reais e decide o acionamento da carga:
 - **Amostra 1 (0,40 V):** Tensão muito baixa ($\leq 0,8\text{ V}$) → Classifica como **LOW** e mantém a **Carga DESLIGADA**.
 - **Amostra 2 (1,40 V):** Tensão no intervalo proibido (0,8 V a 2,0 V) → O sistema detecta ruído elétrico, entra em **ZONA DE INCERTEZA** e emite alerta de **Carga BLOQUEADA por segurança**.
 - **Amostras 3 e 4 (2,80 V e 4,90 V):** Tensões altas e seguras ($\geq 2,0\text{ V}$) → Classifica como **HIGH** e aciona a **Carga LIGADA (LED Ativado)**.

Exemplo Visual da Saída Esperada (para Matrícula terminada em 4):

```
=====
SENAI - Pratica de Embarcados e Eletronica
Digito Matricula: 4
Corrente LED: 14.0 mA | R_LED Calculado: 214.3 Ohms
=====
```

Amostra 1: Vin = 0.40 V | ADC = 82 LSB
-> Estado: LOW | Carga: DESLIGADA

Amostra 2: Vin = 1.40 V | ADC = 286 LSB
-> ALERTA: Zona de Incerteza (0.8V a 2.0V)! Carga BLOQUEADA por segurança.

Amostra 3: Vin = 2.80 V | ADC = 573 LSB
-> Estado: HIGH | Carga: LIGADA (LED Ativado)

Amostra 4: Vin = 4.90 V | ADC = 1003 LSB
-> Estado: HIGH | Carga: LIGADA (LED Ativado)

3. Código-Base

Copie e cole no OnlineGDB o arquivo aula07code.c (anexo)

4. Entregáveis Obrigatórios

- **Código do OnlineGDB:** Copie o seu código completo e funcional.
- **Print do Console:** Captura de tela mostrando o cabeçalho impresso com o seu dígito de matrícula e os resultados das 4 amostras.
- **Questionário Teórico-Prático (Respostas Curtas):** Responda às seguintes questões de forma concisa (2 a 3 linhas por resposta) no documento da entrega:
 - **Zona de Incerteza:** Por que a tensão de teste de 1,4 V foi classificada como *Zona de Incerteza* em vez de acionar a carga como nível lógico alto (HIGH) ou baixo (LOW)?
 - **Proteção de Portas e Lei de Ohm:** O que aconteceria fisicamente com o pino de saída do microcontrolador caso o LED fosse conectado diretamente a ele sem o resistor limitador R_{LED} dimensionado?
 - **Resolução de ADC e Overflow:** Por que a variável `valorADC` deve ser declarada com um tipo de 2 bytes (`uint16_t` / `int`) em vez de 1 byte (`uint8_t` / `byte`), considerando a leitura de 10 bits do microcontrolador?
 - **Acionamento de Potência:** Por que cargas que demandam mais corrente (como motores CC ou bobinas de relés) não podem ser alimentadas diretamente pelo pino digital, exigindo o uso de um transistor BJT operando em saturação?
 - **Gestão de Memória SRAM:** Qual é a vantagem de utilizar a diretiva de pré-processamento `#define TENSÃO_FONTE 5.0` em vez de declarar uma variável global convencional como `float tensaoFonte = 5.0;` em um sistema embarcado?